COMPUTER SCIENCE CAPSTONE PROJECT

CROX

Create, Renovate, Organize, eXplore
Dynamic 2D/3D Web-Based Blueprint Designer

DEVELOPED BY
Ruhan Kailash & Zahill Mishal

COURSE
Computer Science

TECH STACK
Python, Flask, JavaScript, CSS3

DATA ARCHITECTURE
JSON Document Store / NoSQL

---

1. Project Overview

The objective of CROX is to make architectural layout planning highly accessible through a strictly browser-based interface. Traditional CAD software requires heavy installations and steep learning curves. CROX bridges this gap by unifying 2D drawing and 3D visualization into a single, cohesive web workspace.

Core Component	Implementation Details
2D Blueprint Editor	A 20-pixel magnetic grid interface handling custom pointer events for spatial positioning, collision boundaries, and coordinate snapping.
3D Visualization	A live isometric view extruded purely via CSS3 DOM transformations, avoiding the computational overhead of WebGL.
Data Persistence	Flask-managed JSON serialization utilizing cryptographically secure UUIDv4 mappings for multi-floor and multi-project states.

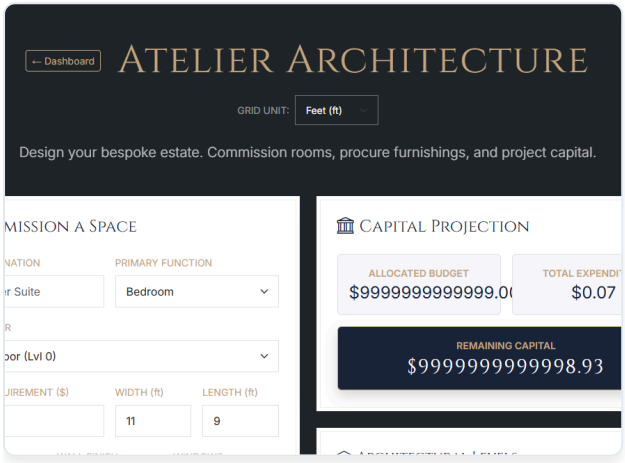
2. Technical Architecture

CROX operates on a lightweight client-server model designed for rapid prototyping and immediate visual feedback. The architecture strictly separates data persistence logic from client-side rendering.

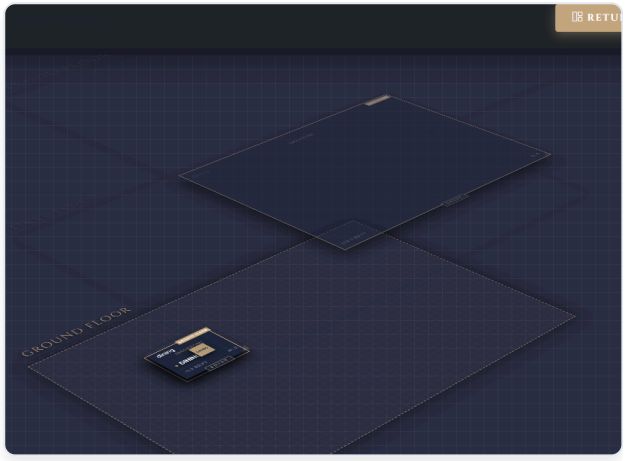
Layer	Technologies Used	Primary Function
User Interface	HTML5, Bootstrap 5	Modal-driven data entry, responsive project dashboard, and layout structure.
Rendering Engine	JavaScript (ES6), CSS3	Real-time DOM manipulation, custom event listeners, and hardware-accelerated 3D matrix transforms.
Application Logic	Python (Flask)	REST API routing, dynamic Jinja templating, and budget/capital arithmetic.
Project Data	JSON Store	Persistent NoSQL-style document storage of hierarchical project objects.

3. The Render Engine: 2D to 3D

Unlike standard web applications, CROX relies heavily on a custom DOM-based rendering engine. Rooms are initiated as flat 2D boundaries. When the user requests a visualization, the application tilts the entire canvas isometrically and extrudes wall depth using CSS pseudo-elements.



2D Planning Mode: Strict 20px grid snapping for precise mathematical dimensions.



3D Isometric Mode: Procedural CSS rendering of depth, walls, and doorways.

```
/* Isometric Projection Matrix */
.blueprint-container-3d-root {
    transform: rotateX(60deg) rotateZ(-45deg);
    transform-style: preserve-3d;
}

/* Procedural Wall Extrusion via Pseudo-elements */
.blueprint-room::after {
    content: '';
    position: absolute;
    width: 10px; height: 100px; /* Z-axis height */
    background: #cccccc;
    transform: rotateY(-90deg) translateZ(10px);
}
```

4. Algorithmic Problem Solving

Challenge A: Event Listener Interference

Context: To support fluid room resizing and dragging, custom pointer tracking was required. However, native browser behaviors (such as text-selection and HTML5 image-dragging) frequently intercepted these events, causing the layout logic to stall.

Solution: The HTML5 Drag-and-Drop API was intentionally bypassed. A custom interaction state machine was built using `mousedown`, `mousemove`, and `mouseup` listeners on the document root. By invoking `Event.preventDefault()` early in the pipeline, native behaviors were suppressed, ensuring frame-perfect tracking and rendering.

Challenge B: Identifier Collisions

Context: In early iterations, room identifiers were generated dynamically based on the length of the room array (e.g., `r_0`, `r_1`). If a room was demolished, the array length decreased, causing subsequent room additions to inherit duplicate identifiers and overwrite existing data.

Solution: The indexing logic was completely refactored in the Python backend. Array-length assignment was deprecated in favor of generating **UUIDv4** (Universally Unique Identifiers) for every project, floor, room, and furniture instance. This eliminated all possibilities of data collision during CRUD operations.

> **Design Outcome:** Transitioning to UUIDs not only prevented data loss but successfully decoupled the frontend rendering logic from the backend storage sequence, resulting in a highly modular and fault-tolerant architecture.